Object Oriented Programming Labs and Exercises

Contents

..... 1

..... 6

01 Revision 7

Objective 7

Part 1 – Make a start! 7

Duration 7

5 minutes 7

Step by step: 7

Creating a new Console application using Visual Studio 2022 7

Building and running the program 8

Part 2 - Creating methods 9

Move your code to a separate class 10

Part 3 - Practice IF statements 11

Part 4 - Practise an iteration statement (loops) 12

..... 13

02 C# Object-Oriented Programming 14

Objective 14

Part 1 – Creating and using reference types 14

Part 2: Create a graphical application 15

Objective 15

Part 1 15

Create a class called Ball 18

Part 2 - Using a struct (Optional) 20

..... 21

03 Statics 22

Objective 22

Part 1 – Step-by-step instructions 22

Registration plate factory 23
Enhancing the Vehicle class 23
Writing code to use the vehicles 24
Part 2 – Using Statics with the graphical project you created earlier 24
..... 26
04 Enums and Strings 27
Objectives 27
Part 1 - Using an enum 27
Part 2 - Using String 28
Part 3 - Using StringBuilder 28
..... 30
05 Collections and Generics 31
Objective 31
Part 1 – Use a List<Shape> 31
Part 2 - Using a Queue and a Stack 31
Part 3 - Using Dictionary<K,V> 32
Step-by-step instructions: 32
..... 34
06 Exceptions 35
Objective 35
Overview 35
Part 1 - Using try/throw/catch in the Account class 35
Part 2 36
..... 37
07 Inheritance 38
Objective 38
Overview 38
Step-by-step: 38
Create a List of Shapes 40

41
08 Polymorphism 42
Objective 42
Overview 42
Part 1 – Racing Cars 42
Scenario 42
Step-by-step: 42
Part 2 – Employee Hierarchy 45
Scenario 45
Designing the hierarchy 45
47
09 Abstract Classes 48
Objective 48
Part 1 48
Part 2 48
Step-by-step 50
53
10 Interfaces 54
Objective 54
Overview 54
Step-by-step: 54
56
11 Introduction to Testing 57
Objective 57
Overview 57
Working with Security class 57
60
12 Accessing Databases Using C# Code 61
Part 1 – Write code to read data from a table 61

Part 2 – Show the selected customer's detail 62

Part 3 – Make changes (if you have time) 62

Revision

01 Revision

Objective

In this series of labs you will :

- Create a program to practise the pre-course studies

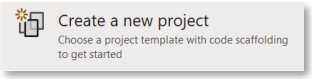
Part 1 – Make a start!

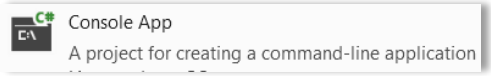
Visual Studio is the development tool used by many C# developers and throughout this course. This lab will show you how to create and run a C# program.

Duration

5 minutes

Step by step:
Creating a new Console application using Visual Studio 2022

1. Launch Visual Studio and then create a new project 

2. Select Console app 

and then click **Next**.
3. Type **C:\Labs** as the location for your project and type **Labs** for the name.
4. You may choose this option if you would like to keep your project and solution file together, but it has little consequence.

5. Click **Next**.
6. Select

for this session so our project layout matches the previous versions of .NET
7. Click **Create**.
8. A starter code is created for you, such as:

```
namespace Labs
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

9. For a cleaner layout and to reduce the number of brackets, hover over the {} icon, then select the first option.

Alternatively, you can change the code manually

```
namespace Labs;

internal class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```

Building and running the program

10. Press **Ctrl+F11** to run the program or click the green **Run** icon.

**** End of Part 1 ****

Part 2 - Creating methods

1. Add a new method to the Program class and just under the main method as **public static int GetInt**(**String** prompt).

   This method displays a prompt (String prompt) and then gets an integer input (using the **Console.ReadLine** method) from the user and returns it as an **int**. (Tip: Use **int.Parse** method.)

2. Create another static method called **string GetString** (**string** prompt). This method displays a prompt (string prompt), then gets a string from the user and returns it as a **string**.

3. Create another method called **Lab1()**.
4. Call the **Lab1()** method in the **Main()** method.
5. Call both methods in the **Lab1()** method to:
   a. Get the user's name
   b. Get the user's age
   c. Display a message that includes the name and age+1 e.g., (if Bob is 20 years old):

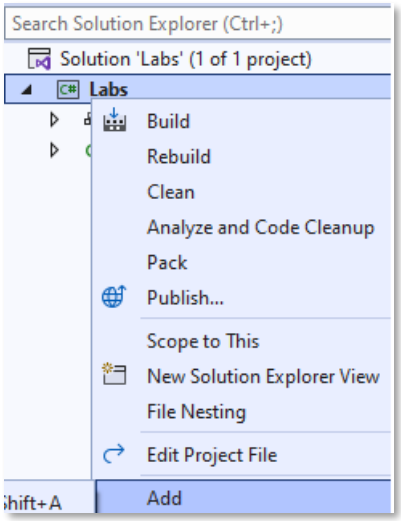
      Dear **Bob** next year you will be **21** years old.

Move your code to a separate class

Does every method have to be in the Program class?
In this section you'll create a new class called Utils and then move the **GetInt** and **GetString** methods to the Utils class.

6. Create a new Class called **Utils**.

   Tip: Right-click on your project and then select the **Add > Class** menus.



7. Transfer the code for the **GetInt** and **GetString** methods to the ***Utils*** class.
8. Remove the **static** word from every method definition.
   We'll discuss **static** method later. The only reason why every method was static was because the **Main()** method in the **Program** class was a **static** method, but we are now free of static main()!
9. At the start of the **Lab1**() method, create an instance of the ***Utils*** class.

   Utils **utils** = **new** ***Utils***();

10. At the start of each method call, add "**utils**". For example:
    Instead of **GetInt()** type **utils.GetInt()**
11. Run the application to make sure everything works.

**** End of Part 2 ****

Part 3 - Practice IF statements

In this section you will calculate the amount of Life insurance

1. Comment out the code in Part 1 & 2.
2. Create a new Class called **LifeInsurance.**
3. Create a new method in the LifeInsurance class called **Calculate** with no parameters which returns a double like public double Calculate()
4. Your **Lab1()** method will call the calculate method like:
   (please comment out any previous code)

   ```
   LifeInsurance life = new LifeInsurance(); Console.WriteLine(life.Calculate());
   ```

5. Calculating life insurance depends on many factors but, for simplicity:
   a. Get the customer's **name** and **age**.
      We only consider ages between 18 and 75 (inclusive).
      Please show a suitable message if age is not within the range and return **-1**.

   b. Find out the number of years the user wishes to keep the life insurance running.
      The duration must be between **10 and 25** (inclusive)
      Older people may not be able to get the terms they ask for. The maximum number of years is calculated as **85 – age.**
      Therefore, a 70-year-old person may only run the scheme for 15 years (**70+15=85)**.

   c. Ask the user for their occupation.
      Apply a **10%** discount for nurses, doctors, teachers, and police officers.

   d. Apply a formula like:
      Minimum payment of **£5 + Age + (Term/10)**
      Apply any discount and then return the result.

***** End of Part 3 *****

Part 4 - Practise an iteration statement (loops)

In this section you will calculate the amount of life insurance.

1. Create a **for loop** in the **Lab1()** method, execute the line `containing life.Calculate();` **3** times.
2. Replace the **for loop** with a **while loop**.
3. Test your code to make sure it works
4. Within the while loop, ask the user if they would like to calculate another quote, and break out of the loop if they say no.

***** End of revision exercises *****

C#
Object-Oriented Programming

02 C# Object-Oriented Programming

Objective

The primary objectives of this lab are:

- to create and use references.
- to consolidate knowledge of passing by value and passing by reference.

Part 1 – Creating and using reference types

1. In the **labs** project, create a class called **Account** with the following fields
   ```
   private int id;
   private string owner;
   private double balance;
   ```

   and methods:
   ```
   void Deposit(double amount) { }
   void Withdraw(double amount) { }
   ```

2. Create a constructor for the **Account** class to set its fields (id, owner, and balance).

3. Modify the **Deposit** and the **Withdraw** methods to make sure no one can Deposit a negative amount of money or Withdraw the money they don't have (in their **balance**).
4. Create a method called **GetDetails**() to return details of the account as a **string** made up of the **id + owner + balance** (put a ',' between these).
5. In the Program class, create a static method called **Lab2()**.
6. Call Lab2() method from within **Main()**.
7. Comment out the call to any other method.
8. In the Lab2() method, create an instance of the **Account** class.

   And then invoke its **Deposit()** and **Withdraw()** methods.
9. Call the Account instance's **GetDetails()** method and then print the result to make sure your code works.

Part 2: Create a graphical application

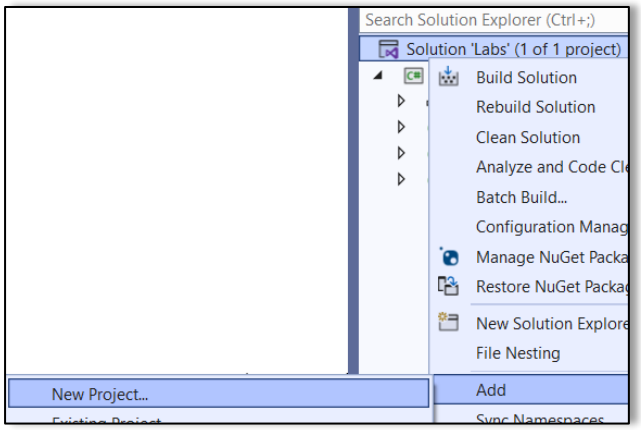
Objective

You will further your understanding of object-oriented programming.
In this lab you'll create a graphical application instead of a Console app.

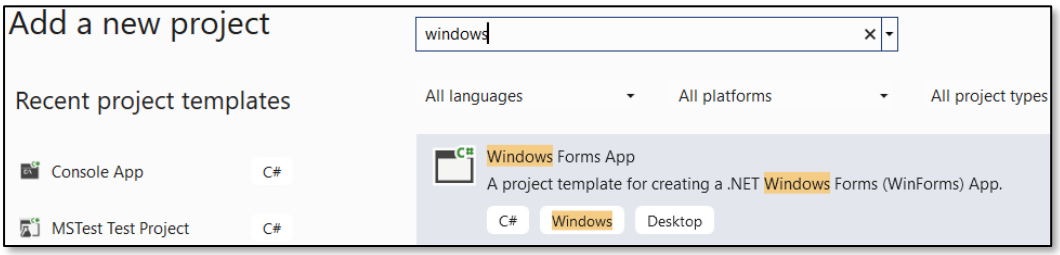
Part 1

You'll soon be creating a graphical application which is new. You'll copy and paste many lines of code. You are not required to understand how the graphics code works but it is not too complex to investigate by yourself.

1. Add a new Windows Form application called **GamesApp** to your labs solution.
   Right-click on the **Labs Solution** and then select the **Add > New project...** menus.



2. Create a new Windows Forms App called **GameApp** by searching for **Windows** and selecting the **Windows Forms App (.Net Framework)** option.



3. Right-click on the newly created GameApp project and then select the **Select as Startup Project...** menu.

4. Have a look at the **Program** class. It creates an instance of a class called **Form1** which represents a Windows Form. You'll use this class to display your graphical objects.

5. You need a canvas upon which you can draw and show your animation. In this case, you'll create a number of ball objects bouncing inside a rectangle (the world).

   You can get creative later and do something else but for now, please follow the steps. The aim is to investigate OO concepts and not concentrate on animation and graphics.

6. Click on the body of the Form and then press **F4** to see its properties (if not displayed already).

7. Click the lightening symbol to see the form's events.

   An **event** is an action which you can handle in code. For example, when a user clicks on a control, the control's click event is fired. In this application, we're after the **Paint** event which is fired when the form needs to be redrawn or repainted.

8. Double-click on the **Paint** event from the list of events. You'll then be redirected to the code behind the form.

9. Type the following code in the Paint event. This code clears the screen and draws a red reactangle. 10,10,600 and 400 are the world's dimensions.

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
        e.Graphics.Clear(Color.Gray);
        e.Graphics.FillRectangle(Brushes.Red, 10, 10, 600, 400);
}
```

10. Run the application to make sure everything works so far.

11. Stop running by closing the Form.

12. Double-click on the Form1 icon in the Solution Explorer window to view it in design mode.

13. Press **F4** and then click to see the form's events.

14. Double click on **Load event** from the list to view an event which fires when the form is loaded for the first time.

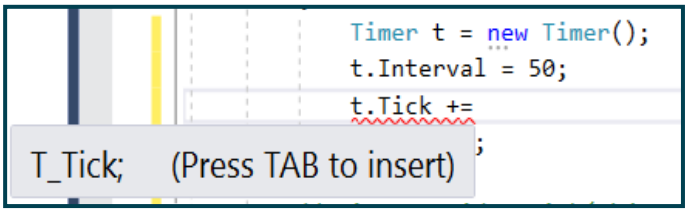
    Type the following code in the *Load event* in order to create a timer which fires every *50* millisecond.

```
private void Form1_Load(object sender, EventArgs e)
{
Timer t = new Timer();
t.Interval = 50;
}
```

You will need to press Ctrl-. (Control dot) On the word **Timer** and

then select `using Timer = System.Windows.Forms.Timer;`

15. Add the following text to the above method `t.Tick +=` and then press the Tab key to create an event handler for the timer's tick event.



16. In the Tick event type: **`this.Invalidate();`**
    This will force the *Paint event* to fire every 50ms.

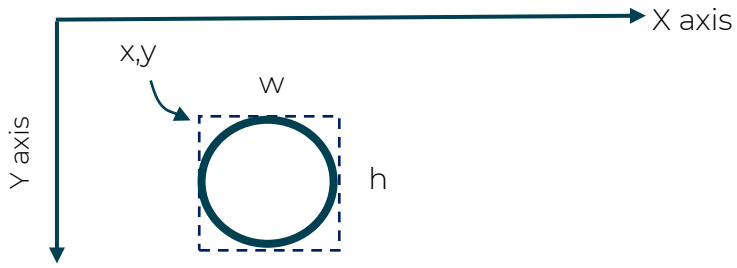
17. Back in the *Form Load* event, activate the timer by adding this line of code:
    `t.Start();`
    From now, all your code for drawing will go into the Paint event.
    Your code will look like:

```
private void Form1_Load(object sender, EventArgs e)
{
    Timer t = new Timer();
    t.Interval = 50;
    t.Tick += T_Tick;
    t.Start();
}
```

Create a class called Ball

18. Create a class called **Ball** with the following fields:

```
public int x, y, w, h;

private int dirX, dirY;
```



x and y represent the top coordinates of the shape (ball) within an upside down coordinate system. dirX and dirY are the amount by which a ball moves in the x and y directions.

Note: We must create these as property procedures but elected to make these public vars for simplicity.

19. Create a constructor to set every field (x,y,w,h,dirX, dirY).

20. To practise constructor chaining, create another constructor to set all the fields except dirX and dirY.

    Using constructor chaining, call the first constructor and pass it dirX=1 and dirY=1.

21. Create a method called **Move()** in the Ball class.
    In this method, increase x by dirX and y by dirY and then follow a series of tests to make sure the ball moves within a designated world area (x=10, y=20, width=600, height=400) which you set in Step 9.

    if (x < 10) then set x=10 and change the sign of dirX
    Tip: dirX = **-**dirX.
    Do the same for the y coordinate (changing dirY of course).

    if (x > 600 – the width of the ball) then set
    x=600 – the width of the ball and reverse the sign of dirX.

22. Do the same for the y coordinate, taking the ball's height into account and the overall height of 400 for the world in which a ball can travel.

23. Create an array field (at the beginning of the class Form1) as: **Ball[] balls;**

```
public partial class Form1 : Form
{
    Ball[] balls = new Ball[3];
```

24. Back in the Form Load event handler, create three instances of Ball and place them inside the **balls** field (see below).

```
private void Form1_Load(object sender, EventArgs e)
{
      balls = new Ball[]
      {
          new Ball(30, 30, 20, 20, 2, 3),
          new Ball(40, 40, 30, 30, 3, 4),
          new Ball(80, 90, 20, 40, 4, 3)
      };

      Timer t = new Timer();
      t.Interval = 50;
      t.Tick += T_Tick;
      t.Start();

}
```

25. In the *Paint event*, create a foreach loop to go through each ball and run its **Move()** method. You will then draw the ball after moving it. Please use the code below to draw a ball:

```
e.Graphics.DrawEllipse(Pens.Yellow, b.x, b.y, b.w, b.h);
// where b is the ball reference you created earlier
```

26. If everything works, you should see all three objects moving around within the world's rectangle.

    If you have a need for speed, then increase the values of dirX and dirY of each ball.

Part 2 - Using a struct (Optional)

System.Drawing.Rectangle is a struct which is defined by Microsoft and can be useful to us in this lab.

This struct can combine four attributes in one. These are X,Y, width and height
It has other properties such as right, bottom, left (or X) and top (or Y).
Here is a sample code to guide you.

```
public class Ball
{
Rectangle world = new Rectangle(10, 10, 600, 400);

    public Rectangle Rec { get; set; }
// …
public Ball(Rectangle rectangle, int dirX, int dirY)
        {
        Rec = rectangle;
        …
        …
        }

        //   In the Ball's move method:

    Rec.Offset(dirX, dirY);

        if(Rec.Right > world.Right)
    {
        dirX = -dirX;
        Rec.Offset(dirX, 0);

        }
}
```

To create a new Ball, do:

```
Ball b = new Ball( new Rectangle(100,50,30,30), 2, 3);
```

**** End ****

Statics

03 Statics

Objective

In this lab you will implement a factory pattern. You will get a 'Registration Plate' from a 'Registration Plate Factory' class.

Part 1 – Step-by-step instructions

In this section you will create a Vehicle class with a static member and instance members and use static members of a Factory class.

1. Open the **Labs** Console solution if not open already.
2. Create a class called **Vehicle** with integer fields called **speed** and **lane**. Also record the distance travelled using an int variable called **distanceTravelled**.
3. Create a constructor to set the initial speed and lane fields.
4. Add two methods called **Accelerate** and **Brake**.
   void Accelerate (**int** amount)

   The Accelerate method will increase the speed, but never to more than **200**!
   It also adds to the **distanceTravelled** (add 1m per unit of speed).

   void Brake(**int** amount)
   // to reduce the speed by the amount

5. It is never a good idea to use magic numbers like 200 (for maximum speed). Replace 200 with a constant like **private const int *MAX_SPEED*** = 200;
6. Add another method as **string GetDetails()** to get the vehicle's details such as speed, lane, distanceTravelled and the plate number (see below).
7. Every vehicle has a registration **plate** which is a complex object.
   Vehicle plates have information about the city, country and the year of registration which should better be defined as a class.
   a. Create a class called ***RegistrationPlate***.
   b. Give this class a String field called ***regPlate***. Provide a public property method to read the value of this field but make the set part private.
   c. Create a constructor to set the *regPlate field*.

8. Add a new field to the **Vehicle** class called ***registrationPlate*** of type *RegistrationPlate*.
   Tip: ***RegistrationPlate*** *registrationPlate;*

   You'll soon set this field using the **Factory pattern**.

Please do not instantiate it yet!

Registration plate factory

9. Let's create a new **factory** class that creates instances of ***RegistrationPlate***.
10. Create a separate class called ***RegistrationPlateFactory***
11. Add the following static array of reg numbers to *RegistrationPlateFactory*.

```
private static String[] regPlates =
 { "MRB1G", "RU16", "TOYS4US", "HNZ57", "PUT3", "JB007" };
```

    Note: The array is static because it will be accessed by a static method.

12. Create a **static** method called **GetNextRegistrationPlate**()
    This method should return an instance of *RegistrationPlate*.

    To create an instance, you'll need the next registration plate from the regPlates array. How would you get the next regPlate index?

    Return null if you run out of regPlates

13. Back in the Vehicle class's constructor, assign a registration plate to the vehicle using the *RegistrationPlateFactory* class.

    Tip:
    this.*registrationPlate* = ***RegistrationPlateFactory.*** **GetNextRegistrationPlate**();

14. Create a new static method called Lab3() in the Program class.
15. Call Lab3() from within Main().
16. Comment out the call to the Lab2() method.
17. In the Lab3() method, create an array of Vehicles, populated with three new vehicles.
18. Print details of the vehicles created in the above step.
    Please make sure the plates are correctly assigned.

Enhancing the Vehicle class

19. How would you count the number of vehicles created?
    I know you created 3! But what if different parts of your program create Vehicles?

    Create a static method called **GetCount()** in the vehicle class to return the count of vehicles. Alternatively, you could create a **static** Property procedure called **Count** to return this value. Just make sure the Set part of the property procedure is **private**.

    Tip: Vehicle's constructor is invoked whenever a vehicle is created.

Please see the slides for more help.

Writing code to use the vehicles

20. Return to the Lab3() method.

    a) The initial speed of the vehicles is **zero.**

    b) Make sure they are placed in lanes 1, 2, and 3.

21. Write a while loop to race the cars (accelerate them).
    until the distance travelled by one of them is more than **1000**.

    You'll accelerate each vehicle by a random number between 1-10 by using the following code:

    ```
    Random rand = new Random();
    int n = rand.Next(10)+1;
    ```

22. Display details of each vehicle as they accelerate on each iteration of the while loop.

23. As soon as a vehicle has travelled 1000m or more, announce it as the winner and break out of the loop.

24. You can get creative and assign a driver to each vehicle
    (create a Driver class).

Part 2 – Using Statics with the graphical project you created earlier

You'll now make use of the static keyword.
Clearly setting the world dimension to x=10, y=10, w=600, h=400 is not a good idea.

All balls should share the same world coordinates. This is a perfect place to use the static keyword.

1. Create four ***static int*** fields in the **Ball** class called **worldX, worldY, worldW** and **worldH** to represent the x, y, width, and height of the world.

2. Create static method to set the static fields for the world coordinates as
   ```
   public static void SetWorld(int wx, int wy, int ww, int wh)
   ```

3. Modify the Ball class code to use the world's coordinates.

   At the beginning of the Form Load event, invoke the SetWorld() method of the Ball class to set the world's coordinate for every Ball.

   ```
   Ball.SetWorld(10, 10, 600, 400);
   ```

4. Modify the form_paint event to use the new world coordinates.

```
private void Form1_Paint(object sender, PaintEventArgs e)
{
      e.Graphics.Clear(Color.Gray);

      e.Graphics.FillRectangle(Brushes.Red, Ball.worldX,
      Ball.worldY, Ball.worldW, Ball.worldH);
      . . .
```

5. Run your code to see if it works.

6. Change the World!
   Let's have some fun with the World's static fields.

   Every time a ball hits the right edge of the world, add 5 to worldX and when a ball hits the left edge of the world, subtract 5 from worldX.

**** End ****

Enums and Strings

04 Enums and Strings

Objectives

- See how to define and use the `enum` keyword to define a new type
- Consolidate on knowledge of the functionality of class `String`
- Introduce the very useful `StringBuilder` class

Part 1 - Using an enum

1. Open the Game project you created previously.
2. Circle/oval is not the only shape! You can give the Ball class a property that dictates its shape. However, the shape must be limited to a list which you define. Let's create this type as an **enum**.
3. Define a new **enum** type called **SHAPE_TYPE** with the following values:

   Rectangle
   ThreeDRectangle
   RoundRectangle
   Oval
   Arc

   ** Please create this enum outside of the Ball class or put it in a new file.
4. Now you can see the name **Ball** does not look like a good choice!
   Please change the name Ball to **Shape** in your project. The best way to do this is to open the Ball class and right-click on the word Ball, then choose the **Rename** menu option. The editor will change all references to Ball.
5. Define a new private field called **shapeType** of type **SHAPE_TYPE** as: **private SHAPE_TYPE shapeType;**
6. Create a property getter for this field (private set). Call this **ShapeType**
7. Set the shapeType's value inside the constructor.
   Tip: Add a parameter of type SHAPE_TYPE to the constructor.
8. Back in the paint method, you must examine the ShapeType property to see what to draw. for example

   if (**ShapeType** == SHAPE_TYPE.**RoundRectangle**)
   e.Graphics.DrawRectangle(Pens.Blue, b.x, b.y, b.w, b.h);
9. Run your application to see different shapes bouncing about!

10. You can also change the colour of your shape by creating a new property of type Color: `public Color Colour { get; set; }`

    Set its value in the Shape's constructor and also create a getter method. You can then use this in paint when drawing a shape using code:

    ```
    Pen p = new Pen(b.Colour);
    e.Graphics.DrawRectangle(p, b.x, b.y, b.w, b.h);
    p.Dispose();
    ```

    Note: **p.Dispose();** is a vital method call, otherwise you will create 1000s of Pen objects and the graphics slow down considerably after a while.

Part 2 - Using String

1. Back in the Labs project, create a new static method in the Program class called Lab4() and then call Lab4() from within Main(). Comment out other code from the previous labs in the Main() method.
2. Expand `Lab4()`, declare a `string` called **name**. Set name to say *Samantha* or any name made up of more than three characters.
3. Display its 3rd character using its index or by using the **Substring** method
4. Display it converted to lowercase and to uppercase.
5. Use a `foreach` loop to iterate over its characters and display each of them tab separated. Throw a line feed after this display.
6. Display whether it `starts with` a `string` of your own choosing.
7. Display whether it `ends with` a `string` of your choosing.
8. Use the `IndexOf` method of string to display the position of the first occurrence of a character that you know is in the `string`, and also, for a character that you know is not in the `string`.
9. Concatenate the 'name' with a surname of your choice to make a variable called '`fullname`', preferably with a space in the middle, then display this `fullname`. Concatenation is okay if it's all done in one statement.
   You can use the + operator or use the C# 6 $"" str*ings.

Part 3 - Using StringBuilder

1. Back in Lab4() method, create a `StringBuilder` object called '**sb**'. Use the constructor that allows you to initialise the object to contain the `String`

   "Bruce Springsteen<space>".
   (You can use the name of your favourite artist instead!)
2. Now use the `Append()` instance method of *`StringBuilder`* to append exactly the text "is the best artist ever" (no error in that!).
3. Use the `ToString()` method of the `StringBuilder` to produce a string that you can display to see the current value of the `StringBuilder`.

You are looking at a strange sentence that needs some amending.

4. Now we would like you to `Insert()` an adjective in front of the word 'artist'. Words like 'greatest' obviously spring to mind, but make your own choice.
5. Now use the `Replace()` method of `StringBuilder` to replace the word 'artist' with a noun of your choice. e.g., 'rock singer'. Display the final result.

**** End ****

Collections and Generics

05 Collections and Generics

Objective

In this lab you will practise using ArrayList, Queues and in a separate task, you'll work with a HashMap.

Part 1 – Use a List<Shape>

1. Open the bouncing shape exercise that you did in the previous lab.
2. Change the array of Shapes (in Game class) to List<Shape>.
   Use the List's **Add** method in the constructor of the Game class to add the three shapes. You can add these during declaration of the List, but better to initialise these in a constructor.
3. Try your code and see if it still works.

Part 2 - Using a Queue and a Stack

1. Back in the **labs** Console project, add a new static method to the Program class called **Lab5()**
2. Call the Lab5() method from Main. Comment out the other calls so we can test the code in this lab.
3. Add another class to the **Labs** console app called **ShoppingBasket**.
4. Add the following fields to the **ShoppingBasket** class:

   ```
   string productName;
   int quantity;
   double price;
   ```

5. Create a constructor for the **ShoppingBasket** class to set the fields.
6. Create a method called **DisplayDetails()** to display the fields' values.
7. In the Program class and above the Main() method, create a static Queue of **ShoppingBaskets** called **baskets**.

   Since we are going to use this queue in Lab5(), it needs to be static.
8. Create a **static** method in the Program class called **Buy()**.
9. Call the **Buy()** method several times from the Lab5() method.
10. Write code in the **Buy()** method to add a few **ShoppingBaskets** to the **baskets** queue. We will process these in another method.
11. Create another static method called **ProcessBaskets**().
    In this method, write code to remove items from the **baskets** queue and call their **DisplayDetails()** method just after they are removed.

This method simulates the processing of shopping baskets for payment and shipping. For simplicity we just investigate the queue actions.

12. Call the **ProcessBaskets()** method in **Lab5()** once.
13. Run and test your code.

Part 3 - Using Dictionary<K,V>

Scenario: A zoo has several animals and is expecting new arrivals soon. They wish to keep track of which animal types they have and record the count of each animal type.

You will create **Dictionary<string, int>**. The key (String) stores an animal's type and a value (Integer) holds the count of the number of said animal in the zoo (e.g., **Lion 3**, **Zebra 2** etc.).

Step-by-step instructions:

1. Create a class called **Zoo**.
2. Create an instance of Zoo in **Lab5()**.
3. Create two methods in the Zoo class called **Open()** and **DisplayAnimalData()** - both with no parameters or return value.
4. Declare and initialise the following fields in the **Zoo** class.

   The following String arrays contain the names of the existing animals and the new animals we wish to add to our zoo.
   The **Dictionary** will keep track of the animals and their count.

   ```
   Dictionary <string, Integer> animals = null;

   string[ ] originalAnimals = {"Zebra", "Lion", "Buffalo"};
   string[ ] newAnimals = {"Zebra", "Gazelle", "Buffalo", "Zebra"};
   ```

5. Instantiate the **Dictionary** of animals (new integers) in the class constructor of the Zoo class or in the Open() method.
6. Create a method in Zoo as **void AddAnimal**(**string** animal).
7. In the Open() method, for each name in the ***originalAnimals,*** Call AddAnimal().
8. Do the same thing as above with the ***newAnimals***.
9. **void AddAnimal(string** animal**)** method should detect if the animal already exists in the ***animals*** Dictionary. If it does, it must increase its count by **1**, otherwise its count must be set to 1.

   Tip: Use the **ContainsKey** method of the Dictionary class to see if the animal exists in the Dictionary.

   You will also use the **Add**() method of the Dictionary class to put an entry

back in the collection and use the Dictionary's indexer (use it like an array) to get the count of an animal type using the name of an animal as the key. For example **animals["Lion"]++;** will increase the count of Lions in the zoo.

10. Call the **DisplayAnimalData()** method from withing **Lab5()** to show all the animal names and their count.

    Tip: Best to use an enhanced for loop.

    This method will display key/value pairs in two columns like this:

    3, Zebra
    1, Gazelle
    1, Lion
    2, Buffalo

11. Run and test your code.

**** End ****

Exceptions

06 Exceptions

Objective

The primary objectives for this lab are to be able to throw and catch exceptions and have a full understanding of the .NET runtime's propagation mechanism for exceptions until a handler is found.

Overview

Firstly, you will handle the exceptions by throwing an exception object in your Account class if balance goes into red.

In a second practical you will enhance the RegistrationPlateFactory class code (created in an earlier lab) so it throws an exception when it runs out of registration plates.

Part 1 - Using try/throw/catch in the Account class

1. Locate the **Account** class you created earlier.

   Add a **Close()** method to the Account class which displays a simple message.
2. Back in the Program class, create a method called **Lab6()**.
3. Call Lab6() from within the Main() method.
4. In the Lab6() method, create an instance of Account with a balance of £100.
5. Withdraw £50 and then display the account's details using the getDetails() method.
6. Withdraw £600 and display the account's details.
   Please note, there is no withdrawal limit in our friendly bank!
7. Fix the **Withdrawal()** method so that it throws an **ArgumentException** when the balance becomes negative.
8. Catch this exception in Lab6() and display a suitable user-friendly message.
9. Build and run the program now. Make sure you get the Account class to throw the exception and make sure it is handled in Lab6().
10. Add a **finally** clause to the exception handler in Lab6() to close the account (using the Close() method) no matter if there is an exception or not.

Part 2

1. Let's go back to the RegistrationPlateFactory code you created in an earlier lab (Lab03).

   The factory class will at some time run out of registration plates.

   - How would you handle this situation?
   - How would you inform the vehicle class that we've ran out of reg plates?
   - How would the vehicle class inform the caller we cannot create a vehicle?

   Implement the required exception mechanisms as best as you can.

   **** End ****

Inheritance

07 Inheritance

Objective

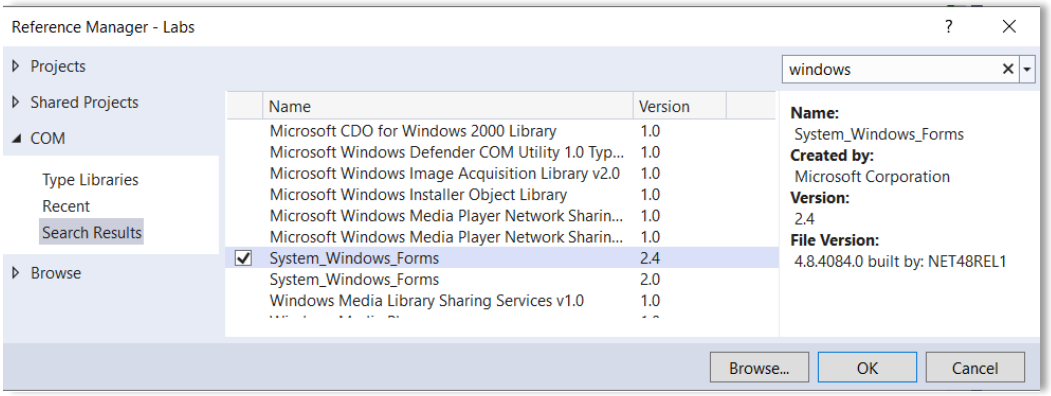
The primary objectives for this lab are to enable you to derive new types and to add specialist functionality.

Overview

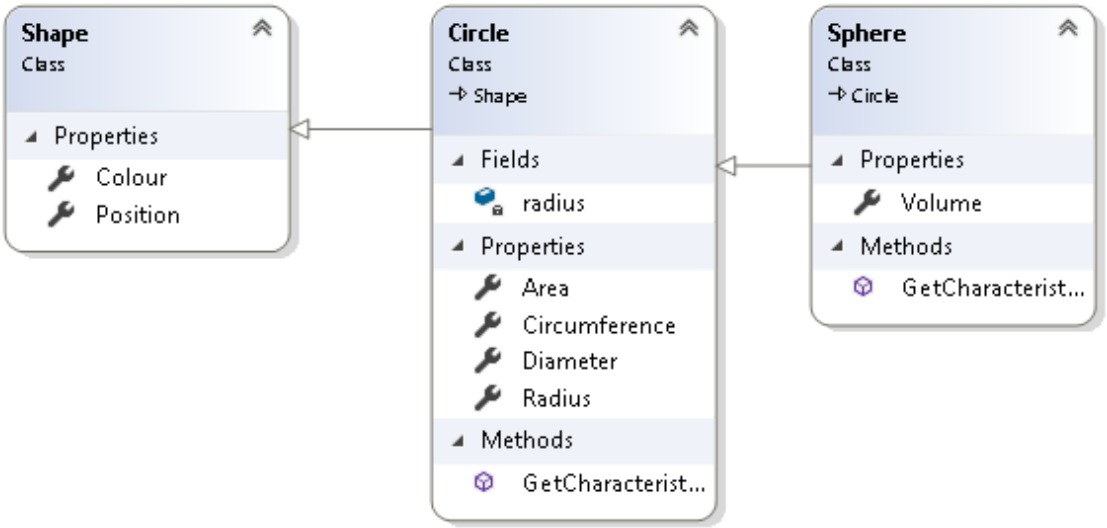
The lab introduces some of the basic concepts of the inheritance story. As mentioned in the associated session, to implement inheritance you must first have a class that provides the fundamental definition or behavior you need. In this lab we will play about with circular shapes.

Step-by-step:

1. Open the **Labs** Console project and make it the *startup project*.
2. The Console lab needs a couple of structures which are included in a Windows Form type of application but not in a Console app. Therefore, we will need to get the libraries for these structures.
    a. Right-click on the Console project's Dependencies in the Solution Explorer window.
    b. Select ***Add Project Reference...***
    c. Click ***COM*** on the left-hand side (**C**omponent **O**bject **M**odel).
    d. Type ***Windows*** in the search box on the right.
    e. Select (check the check box) **System_Windows_Forms**
    f. Click **OK**.



3. Add the following classes using the instruction below.



4. Create a constructor for Shape to set its Colour and Position:
   a. ***Position*** is of type **Point** which is a class with built-in x and y.
   b. ***Colour*** is of type **Color**
      Both Color and Point can be resolved using System.Drawing;

5. As you can see, **Circle** extends **Shape** and **Sphere** extends **Circle**.

   Tip: To calculate the Area/Circumference/Volume you will need **Math.PI** to get the value of **PI**.

   Surface area of a sphere is **4 * PI * R^2** (R to power of 2).

   Volume of a sphere is calculated as **4/3.0 * PI * R^3** (R to power of 3).
   You can use the **Math.Pow**(R, 3) function or R * R * R

   Please note, Area, Circumference, and Volume are defined as Property procedures. Just make sure the **Set** part does not exist by deleting it.

6. Create property procedures for each of the fields (Colour, Radius...) as indicated in the UML class diagram above.

7. The **ToString()** method returns a *String* containing all the attributes of the shape. It will be up to the caller how to display this information.

8. Create a new static method called Lab7() in the Program class.

9. Call Lab7() from within the Main() method.

10. Create a few types of shape in Lab7().

11. Print the characteristics of the Rectangle, Circle, and Sphere objects which

you've created.

Create a List of Shapes

12. Create a **List<Shape>** called **shapes** in the Lab7() method.

13. Add the shapes which you created earlier into the *shapes* List.

14. Create a **foreach** loop to scroll through each shape and print its **colour** and **position**'s X and Y.

    How does this work?!
    How can we store a shape like Rectangle in a List of Shapes?

    All will be revealed in the next chapter!

**** End ****

Polymorphism

08 Polymorphism

Objective

The primary objective for this lab is to enable you to derive new types and to override functionality.

Overview

Read the instructions below and critically evaluate each code sample. Trust nothing and make comments that will be reviewed as a class.

Part 1 – Racing Cars

Scenario

You are going to work with a project which consists of a `Car` class, and a `RacingCar` class. Your code will then fills a `Car[]` array with Car and RacingCar objects.

Your code will then process each `Car`, setting their initial speeds to 60MPH (this is done using a blatant cheat that normal cars don't usually possess – a `GetToSixty()` method).

Your code will then continue by making each `Car` accelerate for 2 seconds before writing their `model` and `speed` to the Console Window.

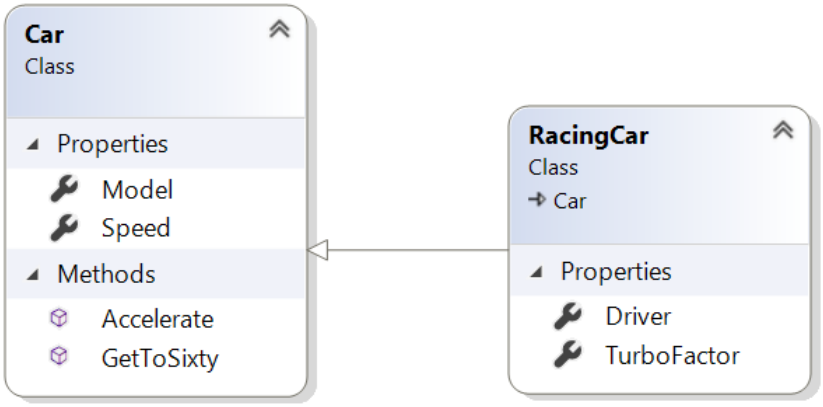
If the car is a `RacingCar` we will need to add some additional code so that a little extra information is written out.

Step-by-step:

1. Open the **Labs** Console project.
2. Create a new static method in the Program class, called **Lab8()**.
3. Call this method from the Main() method, commenting out any other calls.
4. Add a class called **Car** to the Console app project.
5. Add another class called **RacingCar**. This class extends (inherits from) Car
6. The Car and RacingCar classes will have the fields and methods as displayed in the class diagram below.

7. You will also need to add constructors for the two class.

   Please study the diagram below before writing code.
   Note: Any method shown below with the same name as the class is the constructor for that class.



8. Add code to the `Car` class.

   Create getters and setters for the **model** and **speed** fields.
   The **GetToSixty()** method just sets the **speed** to 60.

   The accelerate method should look like **void Accelerate**(**int** *seconds*) and increase the speed by *5 * seconds*.

9. Create a suitable constructor for the `Car` class to set the model.
10. Add a method called GetDetails() to return the speed and the model of the field values.
11. In the `RacingCar` class:
    a. Add a new string property **procedure** called **Driver**.
    b. Add a new public integer property procedure as **TurboFactor**.

12. You'll have to create a suitable constructor for the RacingCar class to set the driver's name as well as the model.

13. Create a method in the RacingCar class called Race() which just prints a message like "Racing!"

14. The **Accelerate()** method will invoke the base class (Car) Accelerate method and then multiply the **speed** by the **turboFactor**.

    Tip: Use the **base.Accelerate()** method.

15. Override the GetDetails of the super class (Car) to return the name of the driver as well.

16. The **Lab8()** method should create an array of cars comprising of a few cars

and racing cars.

17. The Lab8() method should then pass the cars array to a method called `ProcessCars`.

    The `ProcessCars` method currently:

    - gets each `Car` up to 60MPH as a start point.
    - accelerates each `Car` for 2 seconds.
    - displays details of each car.
    - You'll also display the driver's name but only if a `Car` is of type `RacingCar`.

      You'll need to examine the type of car in the array and if it is a RacingCar, cast it to a RacingCar to get the driver's name.

      Tip: Use **is operator** as such: if (car **is** *RacingCar*) {....}
      where '**car'** is a car element in the array of Cars.

18. Replace the **GetDetails()** method with an override of the object class's **ToString()** method. All objects have a ToString() method which you can use to get details of an object.

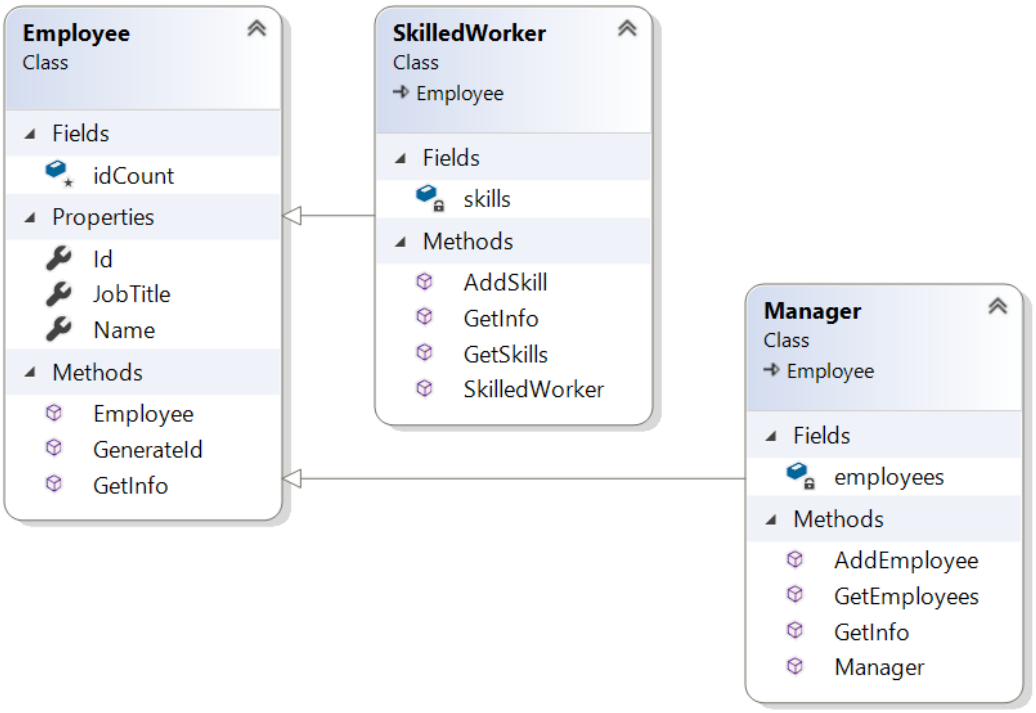
Part 2 – Employee Hierarchy

Scenario

In these labs, you will design and create a class hierarchy for an employee tracking system.

Designing the hierarchy

1. Create the following classes.
   The employee class is provided below. Just copy and paste. We'll need you to concentrate on the other two classes.



To get you started, here is the code for the Employee class:

```
public class Employee
{
    protected static int idCount;
    public int Id { get; set; }
    public string Name { get; set; }
    public string JobTitle { get; set; }

    public Employee(string name, string jobTitle)
    {
        Id = ++Employee.idCount * 10;
        Name = name;
        JobTitle = jobTitle;
    }

    Public override string ToString()
    {
        return $"\n**** Name: {Name}\nJob Title: {JobTitle}\nEmployee ID:
{Id}\n";
    }
```

}

The Manager class holds a List<Employee> called **employees**.

2. The **AddEmployee(Employee emp)** method adds the **emp** object to the **employees** List.

3. The **GetDetails()** method of the Manager should first gather the manager's details (using **base**.GetDetails()) and then use a for-loop to go through the **employees** list in order to call their GetDetails() method.
   It should then return the resulting string.

4. The **SkilledWorker** has a **List**<string> called **skills** which holds the names of skills possessed by a SkilledWorker instance.

5. The **AddSkill(String skill)** method adds the **skill** String to **skills**.

6. In the **Lab8()** method:
   a. create a Manager instance.
   b. create a few regular Employee instances and add them to the manager's employees List.
   c. create a SkilledWorker object with a few skills and then add the instance to the Manager's employee List.
   d. call the manager's GetDetails() method and display the result.

   Question: Can we add a manager instance to a manager's employees?

   **** END ****

Abstract Classes

09 Abstract Classes

Objective

The primary objective of this lab is to provide you with the skills necessary to be able to:

- create and work with abstract classes.
- create classes in a class library type of project.

Part 1

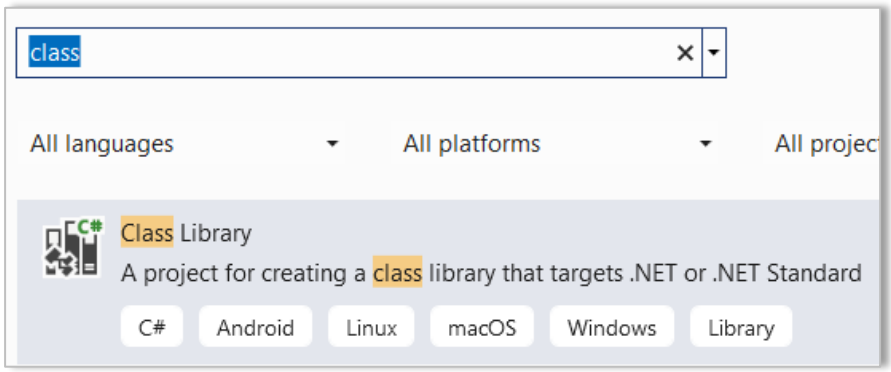
In a previous lab (Lab07 – Inheritance) where you created a number of different Shape classes, it does not make sense to create an object of type Shape. Please mark the Shape class as abstract and then run your code.

Part 2

You are going to put Penguins, Ducks, and Fish in a collection.

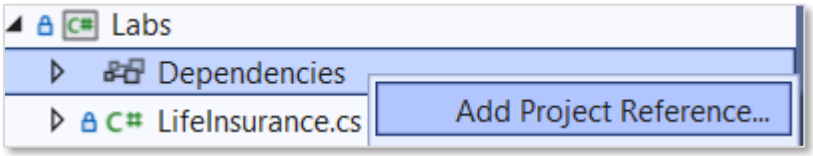
However, you will create these classes in a **Class library** type of project.
As the name implies, this type of project holds classes and compiles the code to *[projectName]***.dll** (**D**ynamic **L**ink **L**ibrary) not *[projectName]***.exe** because the compiled code is loaded into other projects.

1. Open the Labs solution.
2. Right-click on the Solution and then select the **Add** > **New Project**... menus.
3. Type "class" in the search box and then select the **Class Library** project type:



4. Type **QALibrary** as the project name and complete the rest of the steps.
5. Delete the **class1** file as we don't need this class.

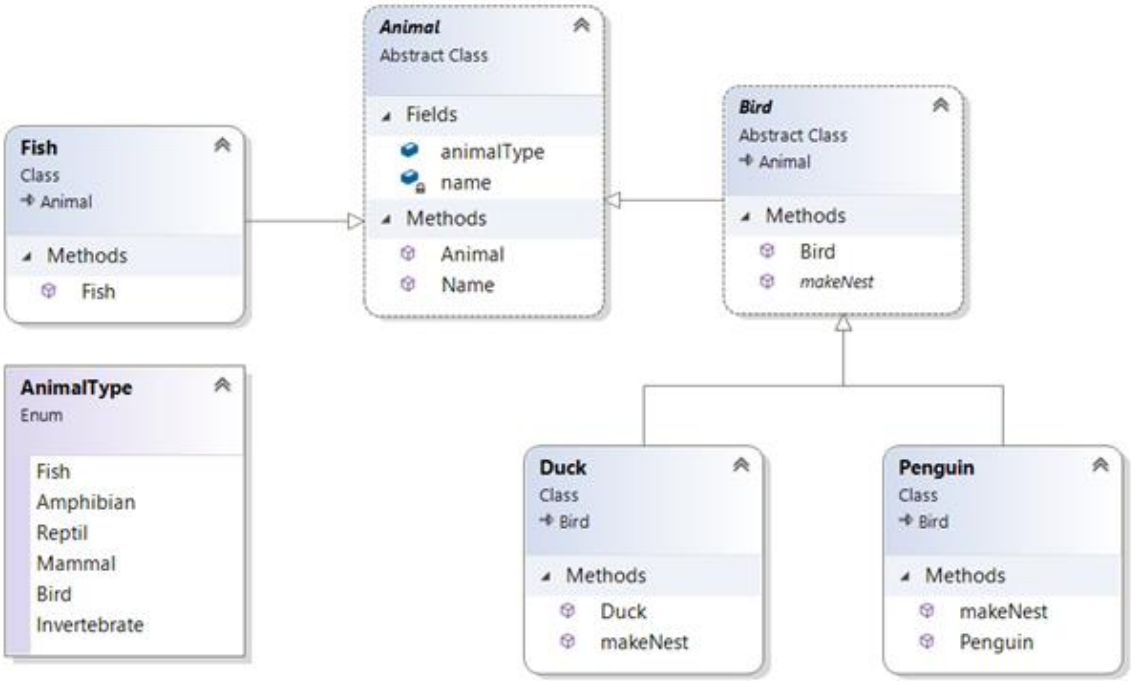
6. The **Labs** Console assembly (project) will use the QALibrary class library and therefore, it needs to reference it.
   Please add a project reference to QALibrary:



7. Create the classes found below (step-by-step section) inside the **QALibrary** project.

8. Penguins and Ducks will be derived from the Bird class.
   We have decided that 'Bird' is too abstract for what we want.

9. The class diagram after completing the lab can be seen below.
   You will find the full description of each class after the UML class diagram.



Step-by-step

1. Add the following classes to the **QALibrary** project:
   a. **Animal**
   b. **Bird**
   c. **Fish**
   d. **Duck**

2. Create an **enum** to define six animal types, as below.
   Please put this enum in its own file.

```
public enum AnimalType {
Fish,
Amphibian,
Reptile,
Mammal,
Bird,
Invertebrate
}
```

3. Add code to the **Animal class:**
   a. Make this class `abstract.`
   b. Add a field to define the type:
      `AnimalType animalType;`
   c. Add a property procedure called **Name**. Make the set part private because once the name is set, it should not change by any other class.
   d. Provide this class with a constructor to set the animal's name.
   e. Feel free to add other properties and methods if you have time.

4. Add code to the **Bird class**.
   a. Make this class also `abstract`
      we want to make different kinds of birds, not a generic bird!
   b. Bird should `Extends` (inherit from) the Animal class.
   c. Create a constructor to set a bird's name.
   d. Set the type of the animal to Bird within the constructor.
   e. Provide it with an abstract method called **MakeNest()** because all birds make their nest in a different way.

      Tip: `public abstract void MakeNest();`

5. Feel free to add other properties and methods if you have time.

6. Write code for the **Duck class**

   a. `Extends` Bird.
      By default a **Duck** will also be an **Animal** (Bird extends Animal).

   b. You'll need to provide a constructor because the super class (Bird) has a constructor with a String parameter to set the name.

   c. Implement the **MakeNest()** method (just print a message).

7. Write code for the **Penguin class:**

   a. `Extends` Bird.
   b. Provide a constructor like the Duck class you created earlier.

8. Create a class called **Fish**.

   a. Set its name and type using a constructor.
      Feel free to give the fish other methods like **Swim()**.

9. Open the class `Program` in the Labs Console project and make sure it is the start-up project (right-click and then select ***Set as Startup Project***).

10. Create a new static method called Lab9() below the Main() method.
11. Call Lab9() from Main().

12. In the **Lab9()** method:
    a. Create a List of **Animal** type called **animals** (List<Animal>).
    b. Create (instantiate) different Animal types and place them in the List of Animals.
    c. Write a 'foreach' loop that iterates over the **animals**.
       This is an example of **Polymorphism**.

       Every Fish, Penguin, and Duck can be referred to as an animal and can therefore be placed in a collection of type **Animal**.

13. Run your code to make sure it works.

14. How would you detect if an animal in the list is a Bird and then call its **MakeNest()** method?

15. Test your code to make sure it works.

16. Add other animal types if you have time.

**** End ****

Interfaces

10 Interfaces

Objective

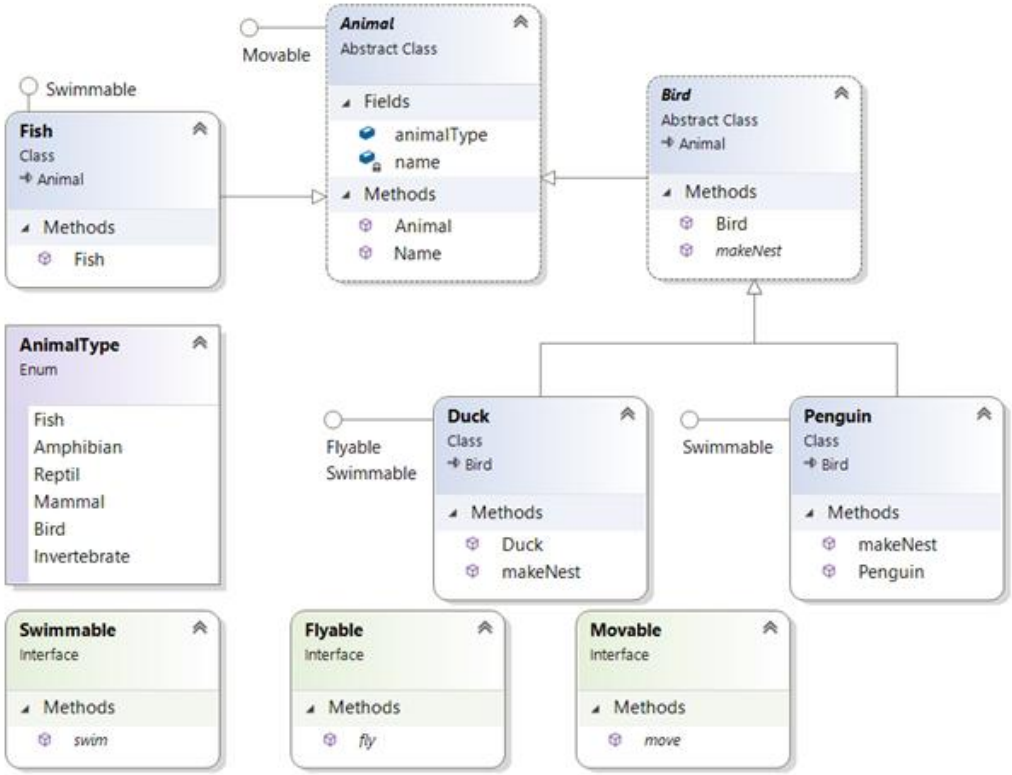
The primary objective of this lab is to provide you with the skills necessary to be able to:

- Define and implement interfaces

Overview

In this lab you will implement a few interfaces to practise this subject.

The class diagram after completing the lab can be seen below.



Step-by-step:

17. Following the code you generated in the previous lab, add the following interfaces to the **QALibrary** project:

    a. **Swimmable** with a method called **Swim**()

    b. **Flyable** with a method called **Fly**()

    c. **Movable** with a method called **Move**()

    See the example below for a movable interface:

    ```
    public interface Movable {
    ```

```
        void Move();
}
```

18. Please view the class diagram above and then implement the interfaces. Just display a message for every implemented method. For example, in the Duck class:

```
public void Move {
  Console.WriteLine("Moving like a Duck!");
}
public void Fly {
  Console.WriteLine("Flying like a Duck!");

}
```

19. Open the Labs Console's Program class and write a few lines of code to test the interfaces' code.
20. Copy the code you wrote for Lab9() method to create a new method called Lab10().
21. Call Lab10() from within the Main() method.
    d. In the `foreach` loop that iterates over the **animals** array, write an if statement to test if an Animal object is `Flyable` and if it is, cast it as **Flyable** and then call its **Fly()** method.
22. Run your code to make sure it works.
23. Inside the loop, test if the Animal object is **Swimmable** and if it is, cast it as Swimmable in order to call its Swim()method.

    Please note, the Penguin class extends to Bird but implements Swimmable which is an attribute of a fish! Interfaces allow us to do things like this and categorise and group widely different types together. Interfaces allow huge flexibility in your design.

**** End ****

Introduction to Testing

11 Introduction to Testing

Objective

The objectives of this practical session are to get you started on unit testing with MS-Test.

Overview

In this lab, we are going to use MS-Test to create unit tests.

You'll test a method that checks if userID and password parameters are valid by following a set of rules.

Working with Security class

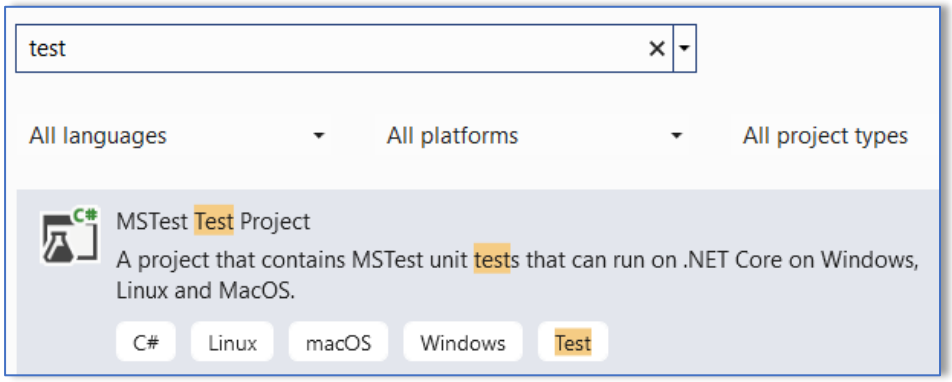
1. Open the **Labs** Console project.
2. Add a new class to this project called **Security**.
3. Add the following code to the Security class:

```
public class Security
{
    public class Security
    {
        public bool Logon(string userId, string password)
        {
            string errorMessage;

            // userId must not be null or spaces. Must be 8 chars minimum.
            // password must contain a digit and an upper case char

            if (userId == null || userId.Trim() == "")
            {
                errorMessage = "UserID may not be null or empty";
                return false;
            }
            else if (password == null || password.Trim() == "")
            {
                errorMessage = "Password may not be null or empty";
                return false;
            }
            else if (!ValidatePassword(password))
            {
                errorMessage = "Password must be min 8 chars with an
                                                uppercase and a number";
                return false;
            }
            else
            {
                errorMessage = "";
                return true;
            }
        }
```

```
        public bool ValidatePassword(String password)
        {
            bool hasUpper = false, hasDigit = false;

            if (password.Length < 8)
                return false;

            foreach (char c in password)
            {

                if (Char.IsUpper(c))
                {
                    hasUpper = true;
                    // continue loop as no character can be both!
                    continue;
                }
                if (Char.IsDigit(c))
                {
                    hasDigit = true;
                }
            }
            return hasUpper && hasDigit;
        }
    }
}
```

You will now test the above logon method against a set of rules which are mentioned as comments in code.

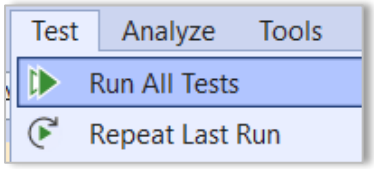
4. First you will create a test case class in an MS-Test project called **LabsTests**.
5. Add a new MS-Test project to the Labs Solution by right-clicking on the solution and then selecting the below options:



6. In the new MS-Test project, set a dependency (reference) to the Labs Console application.
7. **Tip**: Right-click on the **Dependencies** node and select the **Labs** checkbox.
8. You need an example of what a test method looks like before producing your own test methods.
9. Please see the code on the next page for an example.
10. Please use this code to create a few more test methods with suitable

names. Base your tests on the checks made in code. For example, Password should not be blank or null and have a minimum of 8 characters, etc.

11. Run your tests by selecting the **Tests** > **Run All Tests** menus.



12. Replace the test case in this class with the following:

```
using Labs;

namespace LabsTests
{
    [TestClass]
    public class SecurityTests
    {
        [TestMethod]
        public void TestLogonEmptyUserId()
        {
            String userId = "";
            String password = "Freddy99";
            Security s = new Security();
            bool actual = s.logon(userId, password);
            bool expected = false;
            Assert.AreEqual(expected, actual);
            // assertFalse(actual);   // can also use this assert
        }
    }
}
```

**** End ****

Accessing Databases Using C# Code

12 Accessing Databases Using C# Code

Objective

In this lab you will use the ADO Connected methods to extract customer records from the Northwind database and display the data using a Window's form.

Please make sure the Northwind database is installed.
Double-click on the provided Northwind.sql file if you've not installed Northwind.
Alternatively you can choose another database which you have installed in Microsoft SQL server.

In this lab you will display all the **CustomerID, ContactNames** from the **Customers** table. When the user types a CustomerID, your application will display the customer's details.

Part 1 – Write code to read data from a table

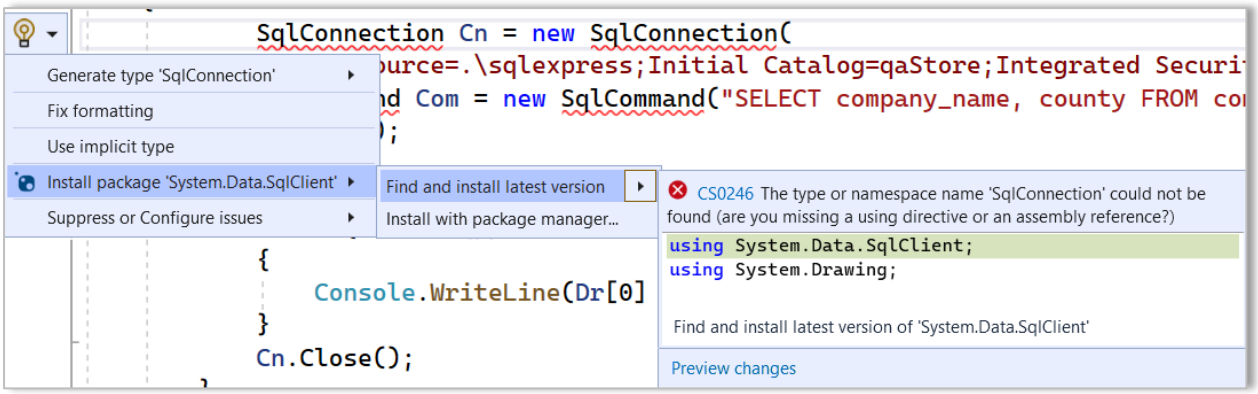
Back in the Labs Console application, create a method called **Lab12()** and then call it from within the Main() method.
A graphical Forms app would be better for this type of app, but we want to keep it simple and focus on the coding part.

Type the following code into the Lab12() method (extracted from the slide code). Please study the code to see what it does.

```
private static void Lab12()
{
  SqlConnection cn = new SqlConnection(

  @"Data Source=.\sqlexpress;Initial Catalog=Northwind;Integrated Security=True");
  SqlCommand com = new SqlCommand("SELECT TOP 10 CustomerID, ContactName FROM customers",
                                          cn);

  cn.Open();

  SqlDataReader dr = com.ExecuteReader();

  while (dr.Read())
  {
     Console.WriteLine(dr[0] + " – " + dr[1]);
  }

  cn.Close();

}
```

There are a lot of compilation errors here because you've not yet imported the necessary libraries.

Press **Ctrl-.** (control dot) on the **SqlConnection** word and install package '**System.Data.SqlClient'** (see below for more help).



Part 2 – Show the selected customer's detail

In the Lab12() method, after calling the cn.Close(), prompt the user to enter a customer's id.

CustomerID in this table is held as a string (like *ANTON*).

After getting an ID from the user, display 4/5 columns' data from the customer table.

We've provided the code for this part at the end of this file but try to do this without looking at the sample code.

Part 3 – Make changes (if you have time)

Create a new method to insert a new customer into the **Customers** table.
Note: CustomerID is alphabetic and should be 5 characters. It must also be unique (is a Primary Key).

Use the code in Lab12() to display the new customer you just inserted into the table.

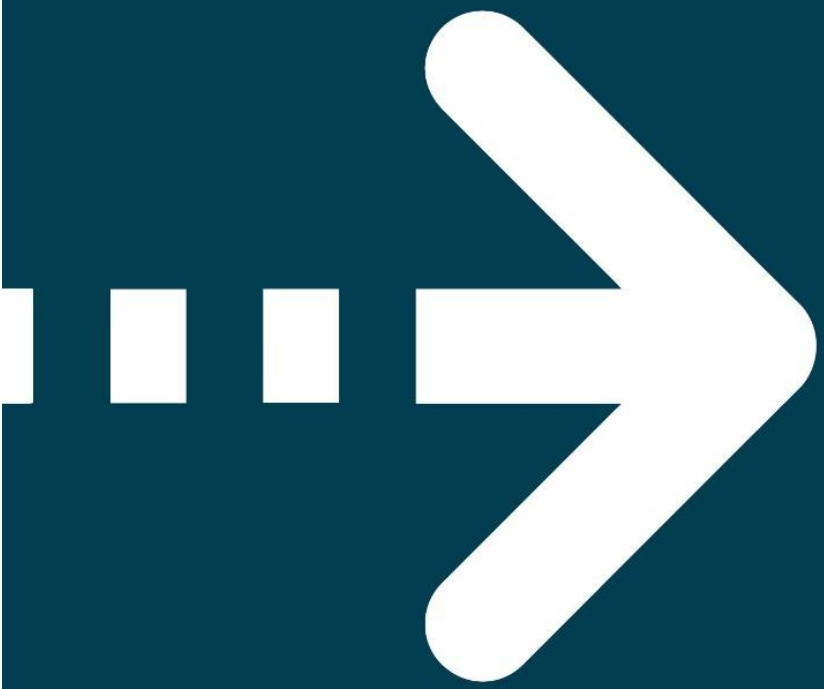
Delete the new customer

See if you can delete the new customer you just added.

Note: You will not be able to delete any other customer because they all have related records in the Orders table and the Referential Integrity constraint stops the deletion of such a record.

** End **

See the completed code sample below.

```
private static void Lab12()
{
  SqlConnection cn = new SqlConnection(
  @"Data Source=.\sqlexpress;Initial Catalog=Northwind;Integrated Security=True");
  SqlCommand com = new SqlCommand("SELECT TOP 10 CustomerID, ContactName FROM customers",
                                          cn);
  cn.Open();
  SqlDataReader dr = com.ExecuteReader();
  while (dr.Read())
  {
     Console.WriteLine(dr[0] + " - " + dr[1]);
  }
  dr.Close();  // connection is left open and only the data reader is closed
  Console.Write("Type a customer ID: ");
  string id = Console.ReadLine();
  com = new SqlCommand($"SELECT * FROM customers WHERE CustomerID='{id}'", cn);
  try {
    dr = com.ExecuteReader();
    dr.Read();
    Console.WriteLine($"{dr[0]}\n{dr[1]}\n{dr[2]}\n{dr[3]}\n{dr[4]}\n{dr[5]}\n");
  }
  catch (Exception ex)
  {
    Console.WriteLine($"No customer found with the id of {id}");
    Console.WriteLine(ex.Message);
  }
  finally
  {
    cn.Close();
  }
}
```

Feel free to refactor the above code by separating out the connection string and the SQL statements into variables to make the code easier to read or by creating new methods to perform these tasks.